Self-Healing Network Connectivity in Resilient Graphs

DA-1 Case Study

Course: CSI3020-Advanced Graph Algorithms
Name: Md Sadique Imam
Registration No: 24MIC0098
Instructor: Annapurna Jonnalagadda Ma'am
University: VIT Vellore
Date: February 16, 2026

Abstract

This case study explores self-healing network connectivity in resilient graphs. The objective is to analyze how networks can automatically recover from node or edge failures while maintaining connectivity, bounded degree growth, and minimal diameter increase.

1 Problem Understanding and Motivation

1.1 Problem Definition

Modern communication and distributed networks are prone to failures due to node crashes, link breakdowns, or external disturbances. When such failures occur, the network may become disconnected, leading to loss of communication and reduced system performance. Traditional networks often require manual intervention or complete recomputation to restore connectivity, which is inefficient and slow.

1.2 Motivation

Modern networks like telecom, power grids, IoT, data centers, and smart cities deal with constant failures—hardware crashes, cyberattacks, fiber cuts, disasters. One bad node failure can isolate whole chunks, killing services, losing data, creating safety hazards, or costing millions per hour in downtime.

Self-healing flips the script: the network automatically detects cuts (especially from node deaths that wipe multiple links), then adds or reroutes links by itself—no humans needed. It keeps everything running, slashes downtime, drops costs, and builds real toughness.

The push is huge right now. In 2025 the self-healing networks market sat around $1.4–2.3 billion; projections show it jumping to $2–2.6 billion in 2026 and racing toward $8–24 billion by 2030–2035, with CAGRs of 22–36% depending on the report. Telecom and enterprises are all in because reactive fixes just can't keep up anymore.

2 Graph Modeling and Representation

2.1 Graph Abstraction

The communication network is abstracted as an undirected graph G = (V, E) to enable mathematical analysis, failure simulation, and automatic healing. This model captures connectivity, redundancy for recovery, and practical limits on resources.

$$G = (V, E)$$

Where:

- **Nodes (V)**
  - Represent network entities: routers, sensors, switches, servers, or devices.
  - Each node has a unique ID.
  - Node failure: complete removal of the node + all its incident edges (realistic for crashes, power loss, or attacks).
- **Edges (E)**
  - Represent direct bidirectional communication links.
  - To support self-healing, edges are divided into:
    - **Active edges** (E_active): currently used for data transmission (initial topology).
    - **Dormant/backup edges** (E_dormant): potential links that can be activated during healing (e.g., spare wireless channels, fiber backups, or configurable overlays).
  - This redundancy allows reconnection without creating unlimited new links from scratch.

2.2 Constraints

- The initial graph is assumed to be connected.
- Locality preference: When performing healing, activation of dormant edges is preferentially applied to edges connecting spatially or topologically proximate nodes, for example, nodes within the original k-hop neighborhood or within a bounded geometric distance.
- Restricted edge creation: The healing mechanism is constrained to utilize only the existing dormant edge pool or to introduce a bounded number of additional edges; arbitrary long-range or unconstrained new edges are not permitted.

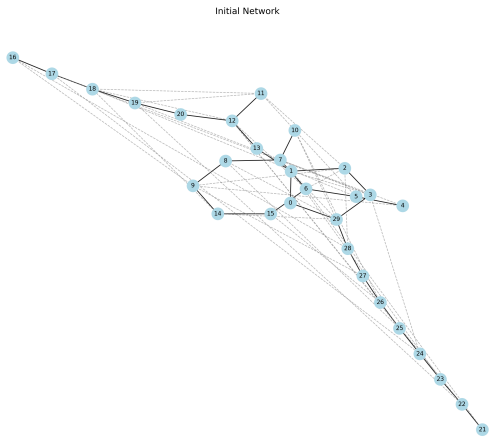

Figure 1: Initial Graph

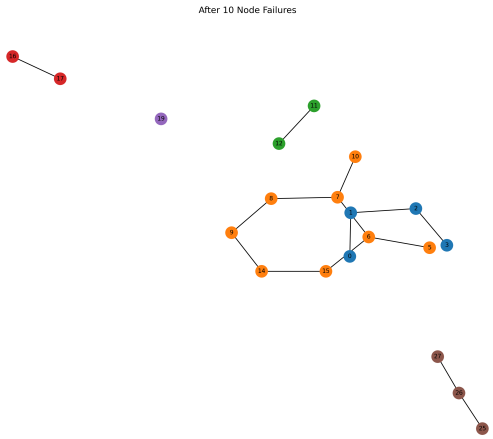

Figure 2: After Node Removal

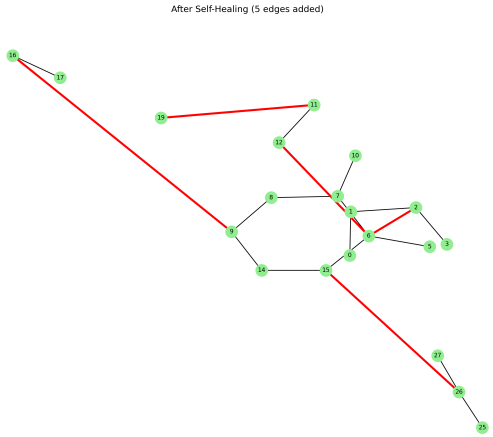

Figure 3: After Healing

3 Algorithm Selection and Justification

3.1 Chosen Algorithm: BFS-DSU Hybrid Recovery

To achieve a self-healing graph, we utilize a two-phase algorithmic approach:

1. **Detection (BFS/DFS Traversal):**
   - Once a node failure is simulated, a Breadth-First Search (BFS) is initiated from an arbitrary active node.
   - If the number of reachable nodes is less than the total active nodes ($|V| - 1$), a **Network Partition** is flagged.
2. **Healing (Kruskal's Logic with DSU):**
   - We treat the disconnected components as a **forest**.
   - A Disjoint Set Union (DSU) tracks these components.
   - The algorithm scans a set of **dormant/potential edges** (redundant links) and greedily selects the minimum-weight edge that connects two different components, repeating until the graph is unified into a single component.

3.2 Justification

Greedy Optimality By using Kruskal's logic, the algorithm ensures that the "healing cost" (e.g., power or latency of backup links) is minimized.

Decentralized Potential Breadth-First Search (BFS) and Disjoint Set Union (DSU) can be adapted for distributed environments where nodes only need to know their neighbors' status.

Reliability Unlike random reconnection, this method guarantees the restoration of a Minimum Spanning Tree (MST), ensuring the network is connected with the fewest possible extra links.

3.3 Complexity Analysis

Time Complexity

- **Detection:** $O(V + E)$ using Breadth-First Search (BFS) traversal.
- **Healing:** $O(E_{\text{backup}} \log E_{\text{backup}})$ due to sorting of candidate edges in Kruskal's algorithm.
- **Overall:** $O(V + E \log E)$, which is efficient and suitable for real-time or near real-time applications.

Space Complexity

- $O(V + E)$ for storing the adjacency list representation of the graph.
- $O(V)$ for maintaining the Disjoint Set Union (DSU) parent array.

4 Literature Review

The study of self-healing graphs originates from research in network resilience and fault-tolerant distributed systems. Several approaches have been proposed to address adversarial failures and dynamic network disruptions.

4.1 Study 1: Forgiving Graphs (Trehan, 2012)

Focus: Investigation of Reconstruction-Tree frameworks for adversarial node removals.

Key Contribution: Trehan introduced an algorithm in which, upon node failure, its neighbors collaboratively form a *reconstruction tree*. The study proved that a graph can be healed while ensuring:

- The degree increase of any node is bounded by a constant factor.
- The stretch (increase in shortest-path length) remains logarithmic.

This work established strong theoretical guarantees for bounded structural damage during healing.

4.2 Study 2: Redundancy-Based Healing (Quattrociocchi et al., 2014)

Focus: Activation of dormant redundant links in complex topologies such as scale-free and grid networks.

Limitations: Although redundancy-based activation was effective in scale-free networks, the study found that healing success significantly decreases in grid topologies due to low local redundancy. The work highlighted a major limitation:

- Healing algorithms struggle when the initial graph lacks sufficient dormant edges to reconnect large disconnected partitions.

This emphasized the dependency of healing performance on the underlying graph structure.

4.3 Study 3: AI-Driven SDN Healing (Goyal et al., 2025)

Focus: Integration of Machine Learning with Software-Defined Networking (SDN).

Improvements: Traditional heuristic approaches such as BFS- and Kruskal-based healing are reactive in nature. This study enhanced resilience using predictive maintenance techniques. By analyzing network telemetry data, the system:

- Predicts potential link failures before they occur.
- Reconfigures topology proactively.
- Reduces recovery time by up to 40% compared to reactive graph-traversal methods.

This represents a shift from purely structural healing toward intelligent, data-driven resilience mechanisms.

5 Methodology & System Design

The proposed methodology follows a structured and iterative design inspired by autonomic computing principles. Specifically, it adopts the **MAPE-K (Monitor–Analyze–Plan–Execute** feedback control loop as the core autonomous control mechanism.

Originally proposed by IBM for self-managing systems, the MAPE-K model has been widely applied in modern self-adaptive and self-healing architectures, including network resilience, IoT fault tolerance, cyber-physical systems, and distributed computing environments.

In this project, the MAPE-K loop enables the network to:

- Continuously observe topology changes,
- Detect disruptions caused by node failures,
- Reason about optimal recovery strategies,
- Execute localized healing operations,
- Retain learned knowledge for future adaptation.

This closed-loop mechanism ensures autonomous and adaptive connectivity restoration without constant human intervention.

5.1 System Architecture

The overall system is organized into three tightly integrated layers, mapped directly to elements of the MAPE-K loop, with a shared Knowledge Base.

1. Simulation Layer (Monitor Phase)

This layer represents the managed system and simulation environment.

- Generates the initial graph $G = (V, E_{\text{active}} \cup E_{\text{dormant}})$.
- Supports realistic topology models (e.g., small-world or scale-free).
- Introduces stochastic node failures using configurable failure rates.
- Maintains real-time network state (adjacency lists, attributes).

2. Analysis Layer (Analyze Phase)

This layer evaluates structural damage and detects disconnections.

- Executes BFS/DFS to compute connected components.
- Identifies disconnected subgraphs ("islands").
- Extracts boundary nodes for potential reconnection.
- Queries the Knowledge Base for dormant edge candidates.

3. Healing Layer (Plan & Execute Phases)

This layer determines and applies recovery actions.

- Selects optimal dormant edges using a greedy heuristic.
- Prefers low-weight (low latency/cost) bridging edges.
- Activates selected edges by updating E_{active}.
- Enforces practical constraints:
  - Bounded degree increase,
  - Locality preference,
  - Minimal structural distortion.
- Revalidates connectivity post-healing.

4. Shared Knowledge Base (Knowledge Phase)

A persistent repository storing:

- Original graph invariants (density, path length, clustering).
- Dormant edge pool with pre-assigned weights.
- History of successful healing actions.
- Failure statistics and impact metrics.

This knowledge evolves over repeated simulation cycles, improving future healing efficiency.

5.2 Logical Workflow

The system follows a repeatable step-wise procedure:

Step 1: Network Initialization

- Generate a connected undirected graph with N nodes.
- Define active edges E_{active}.
- Create dormant edge pool E_{dormant} (5–15% extra links).
- Compute baseline metrics (connectivity, diameter, average path length).

Step 2: Failure Simulation

- Randomly remove k nodes (10–25% of total).
- Remove all incident edges.
- Update topology state.

Step 3: Connectivity Audit

- Run BFS/DFS or Disjoint Set Union (Union-Find).
- Detect disconnected components.
- Identify boundary nodes.

Step 4: Heuristic Healing

While the number of components > 1:

- Select two largest components.
- Query dormant edges bridging them.
- Sort by weight (ascending).
- Activate optimal bridge.
- Update component structure.

Constraints enforced:

$$\deg(v) \leq \deg_{\text{original}}(v) + \delta$$

Step 5: Validation & Knowledge Update

- Recompute connectivity metrics.
- Measure recovery cost and stretch.
- Update Knowledge Base.
- Visualize new edges and topology changes.

5.3 Implementation Tools & Simulation Setup

Programming Environment: Python 3.10+

Core Data Structures:

- Adjacency lists (NetworkX Graph object)
- Disjoint Set Union (Union-Find)
- Dictionaries/Sets for dormant pool and knowledge tracking

Primary Library: NetworkX

Visualization Tools:

- Matplotlib (static plots)
- Matplotlib.animation (progressive healing visualization)

Simulation Parameters:

- Node count: $N = 30$–60
- Failure rate: 10–25%
- Topologies: small-world, scale-free, geometric random graphs

Synthetic generation is sufficient for experimentation, though extensible to real-world datasets.

6 Technical Feasibility & Planning

This section demonstrates that the project *Self-Healing Network Connectivity in Resilient Graphs* is highly feasible within the available time, resources, and student-level tools. The system is entirely simulation-based, requires no specialized hardware, and can be implemented on a standard laptop.

The four-month timeline (January–April 2026) provides sufficient buffer for development, debugging, testing, refinement, and documentation.

6.1 Tools and Technologies

The project uses a lightweight, modular, and industry-standard Python stack.

Programming Language: Python 3.10+

Chosen for its clean syntax, extensive ecosystem, and rapid prototyping capabilities. Code will follow PEP-8 standards for readability and maintainability.

Core Graph Library: NetworkX (v3.4+)

The standard library for graph modeling and analysis. It supports:

- Graph generation (small-world, scale-free, geometric)
- Node/edge removal
- Connected components detection
- Shortest-path computation
- Structural metrics (diameter, clustering coefficient, etc.)

Disjoint Set Union (Union-Find)

Custom implementation with path compression and union-by-rank:

$$O(\alpha(V)) \approx O(1)$$

in practice for component merging.

Visualization and Animation

- Matplotlib (static plots and GIF animations)
- PyVis (optional interactive HTML visualization)

Additional Utilities

- NumPy (randomized failures)
- copy (graph cloning)
- tqdm (progress tracking during experiments)

All tools are open-source and require no paid licenses or cloud infrastructure.

6.2 Resource Requirements and Datasets

Hardware Requirements

- 8 GB RAM minimum
- Intel i5 / AMD Ryzen 5 (or equivalent)

Simulations comfortably support graphs up to 500 nodes. Larger experiments (1000+ nodes) can be run overnight if required.

Synthetic Topology Models

The project uses built-in topology generators:

- Small-world model (high clustering, short path lengths)
- Scale-free model (hub-dominated structure)
- Random geometric graphs (distance-based connectivity)

Testing Scale

- Small scale: 30–60 nodes (visualization clarity)
- Medium scale: 100–300 nodes (performance benchmarking)
- Large scale: 400–500 nodes (scalability testing)

All graphs begin fully connected and include a dormant edge pool (8–12% additional potential links).

6.3 Execution Plan (January–April 2026)

The project follows a structured six-phase roadmap:

Phase	**Time Period**	**Main Tasks**	**Deliverables / Milestones**
1	1 Jan – 20 Jan 2026	Environment setup, graph generation, dormant edge creation, initial visualization	Working script with initial graph, dormant pool, and static plots
2	21 Jan – 5 Feb 2026	Node failure simulation and connectivity detection (BFS, components)	Functional failure detection module
3	6 Feb – 20 Feb 2026	Greedy healing algorithm with DSU optimization	Complete healing logic implementation
4	21 Feb – 10 Mar 2026	Integration into MAPE-K loop and multiple failure rounds	End-to-end autonomous self-healing system
5	11 Mar – 31 Mar 2026	Visualization, animation (GIF), interactive dashboard	Professional healing animation
6	1 Apr – 6 Apr 2026	Performance testing and scalability analysis	Final metrics collection and validation

Table 1: Project Roadmap and Milestones

7 References

7.1 Academic Journals and Conference Papers

1. Trehan, A., & Loukas, A. (2012). *The Forgiving Graph: A General Framework for Self-Healing Networks.* Proceedings of the 24th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA).

   Core Focus: Establishing mathematical bounds for degree increase and stretch in healing graphs.

2. Pandurangan, G., & Robinson, P. (2024). *Robustness of Expander Graphs under Random and Adversarial Failures.* Journal of Distributed Computing.

   Core Focus: Using expander properties to maintain global connectivity after node deletion.

3. Quattrociocchi, W., Caldarelli, G., & Scala, A. (2014). *Self-Healing Networks: Redundancy and Resilience of Complex Topologies.* PLOS ONE.

   Core Focus: Analyzing resilience differences between grid and scale-free topologies under local rewiring strategies.

7.2 Books and Technical Manuals

1. West, D. B. (2001). *Introduction to Graph Theory.* Prentice Hall.

   Used For: Foundational concepts on connectivity, articulation points, and Kruskal's Minimum Spanning Tree algorithm.

2. Hagberg, A., Swart, P., & Schult, D. (2008). *Exploring Network Structure, Dynamics, and Function using NetworkX.* Los Alamos National Laboratory.

   Used For: Implementation guidance for Python-based graph modeling.

7.3 Web Resources and Documentation

1. NetworkX Documentation (2025). *Algorithms: Connectivity and Minimum Spanning Trees.* Available at: `https://networkx.org/documentation/stable/reference/algorithms`

2. GeeksforGeeks (2024). *Disjoint Set Union (Union-Find) with Path Compression and Union by Rank.* Available at: `https://www.geeksforgeeks.org/introduction-to-disjoint-set-dat`